Week #8: Eigenvalue Problems

Francisco Richter Mendoza
Numerical Methods

June 22, 2025

Overview

Eigenvalue problems appear in vibration analysis, stability of differential equations, principal component analysis, PageRank, and quantum mechanics. The problem is to find scalars λ and vectors x such that $Ax = \lambda x$.

Unlike linear systems, eigenvalue problems are nonlinear and require iterative methods. We start with the power method and move to the QR algorithm, which is the standard approach for dense eigenvalue problems. We also cover sensitivity analysis to understand when computed eigenvalues are reliable.

The second half covers the Singular Value Decomposition (SVD), which extends the eigenvalue concept to rectangular matrices. SVD is used in data compression, least squares problems, and forms the basis for techniques like latent semantic analysis and attention mechanisms in transformers.

---

1 Mathematical Foundation of Eigenvalue Problems

1.1 Basic Definitions and Properties

The fundamental concept we are studying is the existence of invariant directions under linear transformations. When a matrix A acts on a vector space, most vectors change direction. However, certain special vectors, called eigenvectors, are merely scaled by the transformation.

Definition 1.1 (Eigenvalue Problem)**.** For a matrix $A \in \mathbb{R}^{n\times n}$, the eigenvalue problem is to find scalars λ (eigenvalues) and nonzero vectors x (eigenvectors) such that:

$$Ax = \lambda x$$

To find these eigenvalues, we can rewrite the equation as $(A - \lambda I)x = 0$. For a non-trivial solution $x \neq 0$ to exist, the matrix $(A - \lambda I)$ must be singular. This condition leads us to the characteristic polynomial.

Theorem 1.1 (Characteristic Polynomial)**.** *The eigenvalues of A are the roots of the characteristic polynomial:*

$$p(\lambda) = \det(A - \lambda I)$$

This is a polynomial of degree n, so A has exactly n eigenvalues (counting multiplicities) in $\mathbb{C}$.

The set of all eigenvalues is a fundamental signature of the matrix, governing its behavior in dynamic systems and iterative processes. We formalize this collection and related concepts below.

Definition 1.2 (Spectral Properties)**.** For matrix A:

- **Spectrum**: $\sigma(A) = \{\lambda : \det(A - \lambda I) = 0\}$
- **Spectral radius**: $\rho(A) = \max_{\lambda \in \sigma(A)} |\lambda|$
- **Algebraic multiplicity**: Multiplicity of λ as a root of the characteristic polynomial
- **Geometric multiplicity**: $\dim(\ker(A - \lambda I))$

Example 1.1 (Experiment: Sensitivity Analysis)**.** **Setup:** We compare two 50×50 matrices with the same eigenvalues $\lambda_i = i$: a symmetric matrix A_{sym} (well-conditioned) and a non-normal matrix A_{non} (ill-conditioned). We add random perturbations E of varying norm ϵ and measure the maximum shift $\max_j \min_i |\tilde{\lambda}_j - \lambda_i|$.

Observation: The figure below shows a dramatic difference in sensitivity. For the symmetric matrix, the eigenvalue errors $|\Delta\lambda|$ are of the same order as the perturbation ϵ (slope 1). For the non-normal matrix, the errors are orders of magnitude larger. This amplification occurs because the eigenvectors of the non-normal matrix are nearly parallel (ill-conditioned). Mathematically, the sensitivity is bounded by $\kappa(V)\epsilon$, where $\kappa(V) = \|V\|\|V^{-1}\|$ is the condition number of the eigenvector matrix. For symmetric matrices, V is orthogonal, so $\kappa(V) = 1$, resulting in perfect stability.

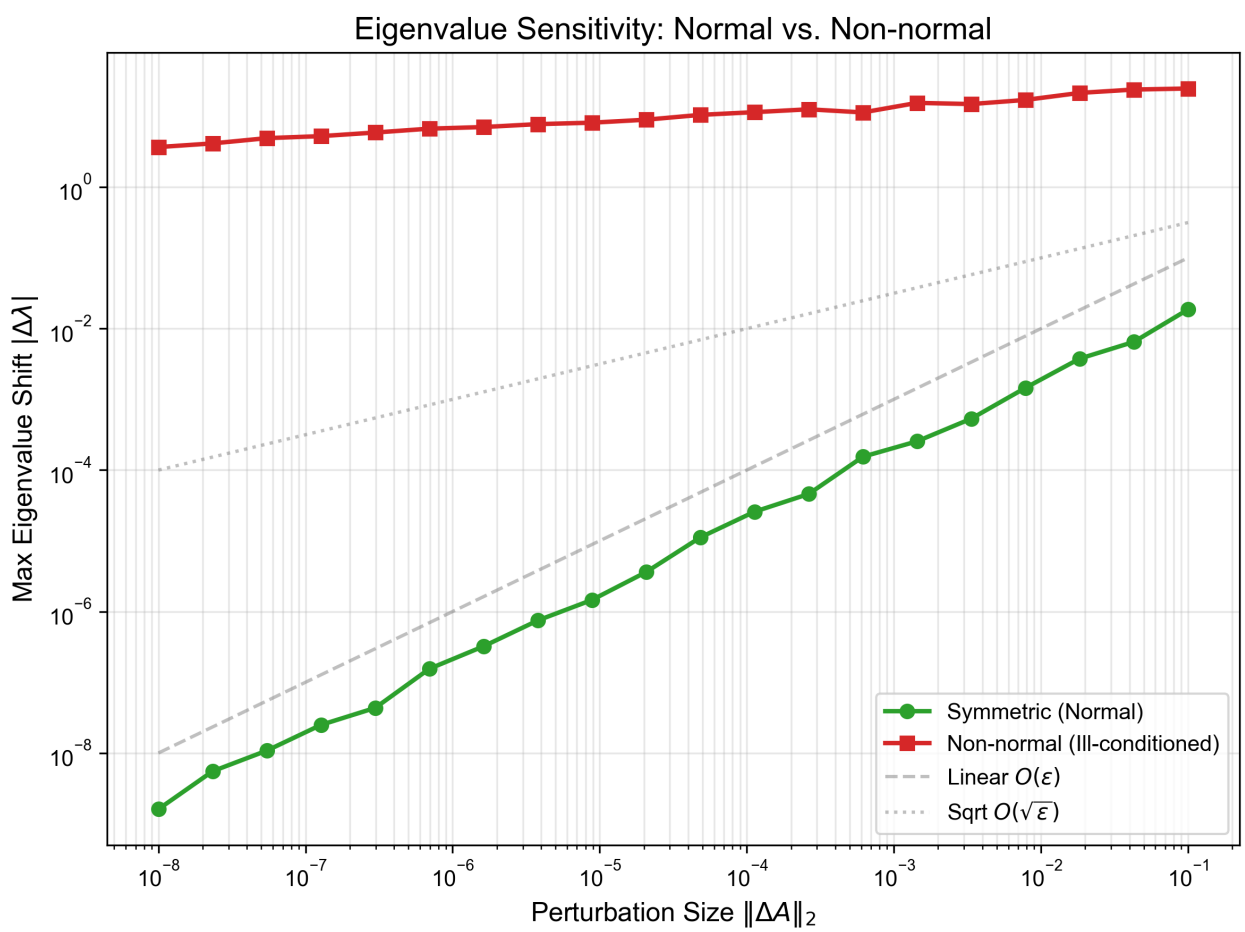

Figure 1: Eigenvalue sensitivity: Comparison of well-conditioned symmetric matrices vs. ill-conditioned non-normal matrices. Note how non-normal matrices can have eigenvalue shifts orders of magnitude larger than the perturbation size.

1.2 Special Matrix Classes

Certain classes of matrices appear frequently in physical applications and possess nice spectral properties that we can exploit numerically. Symmetric matrices, for instance, arise in optimization and physics (e.g., the Hessian matrix, stiffness matrices) and guarantee real eigenvalues.

Theorem 1.2 (Symmetric Matrices). *For symmetric $A \in \mathbb{R}^{n\times n}$:*

- *All eigenvalues are real*
- *Eigenvectors corresponding to distinct eigenvalues are orthogonal*
- *A is diagonalizable: $A = Q\Lambda Q^T$ where Q is orthogonal*

A broader class of matrices that includes symmetric, skew-symmetric, and orthogonal matrices is the class of normal matrices. These are the matrices that commute with their transpose.

Theorem 1.3 (Normal Matrices). *A matrix A is normal if $A^TA = AA^T$. Normal matrices satisfy:*

- *A is unitarily diagonalizable: $A = Q\Lambda Q^*$*
- $\|A\|_2 = \rho(A)$
- *The eigenvectors form an orthonormal basis*

2 The Power Method and Variants

The power method is a simple technique for computing the dominant eigenvalue and eigenvector of a matrix. It is based on the idea that repeated multiplication of a random vector by the given matrix yields vectors that eventually tend towards the direction of the dominant eigenvector.

But why should we ever want to compute the dominant eigenvector of a matrix? Here is a motivating case study.

2.1 Motivation: Google's PageRank

Example 2.1 (Google's PageRank). Google's search engine uses an algorithm called **PageRank** to rank webpages. This algorithm amounts to a computation of the dominant eigenvector of a large and very sparse matrix.

Given a network linkage graph with n nodes (webpages), the importance of a webpage is given by the number and importance of pages that link to it. Mathematically, suppose the importance, or *rank*, of page i is given by x_i. To determine x_i, we look at all pages j that link to i. If page j points to N_j pages and has rank x_j, it contributes x_j/N_j to the rank of page i:

$$x_i = \sum_{j\in B_i} \frac{1}{N_j} x_j, \quad i = 1, \dots, n.$$

where B_i is the set of pages linking to i. In matrix notation, this is exactly an eigenvalue problem $Ax = x$, where A is a column-stochastic matrix with entries $a_{ij} = 1/N_j$ if $j \to i$ and 0 otherwise.

We are looking for the eigenvector corresponding to $\lambda = 1$. But why does such an eigenvalue exist? Since every column of A sums to 1 (representing probabilities), we have $\mathbf{1}^TA = \mathbf{1}^T$, where $\mathbf{1}$ is the vector of all ones. This means that 1 is an eigenvalue of A^T with eigenvector $\mathbf{1}$. Since a matrix and its transpose have the same eigenvalues, 1 must also be an eigenvalue of A. Furthermore, for stochastic matrices, $|\lambda_i| \le 1$ for all i, so $\lambda = 1$ is the dominant eigenvalue.

Why do we care about the dominant eigenvector? In the "random surfer" model, the PageRank vector x represents the steady-state probability distribution. That is, x_i is the probability that a random surfer, after clicking many links, lands on page i. This steady state corresponds exactly to the eigenvector for $\lambda = 1$, which is the only state that persists (does not decay) as $k \to \infty$.

2.2 Developing the Power Method

Suppose the eigenvalues of A are $\lambda_1, \ldots, \lambda_n$ with $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$. Let $v^{(0)}$ be an initial vector. We can express it as a linear combination of eigenvectors x_j:

$$v^{(0)} = \sum_{j=1}^{n} \beta_j x_j$$

Multiplying by A repeatedly (k times):

$$A^k v^{(0)} = \sum_{j=1}^{n} \beta_j \lambda_j^k x_j = \lambda_1^k \left(\beta_1 x_1 + \sum_{j=2}^{n} \beta_j \left(\frac{\lambda_j}{\lambda_1} \right)^k x_j \right)$$

Since $|\lambda_j/\lambda_1| < 1$ for $j \neq 1$, the terms in the sum decay to zero as $k \to \infty$. Thus, $A^k v^{(0)}$ aligns with x_1.

2.3 The Power Method Algorithm

To prevent overflow/underflow, we normalize the vector at each step.

Definition 2.1 (Algorithm: Power Method)**.** Input: Matrix A, initial guess $v^{(0)}$. For $k = 1, 2, \ldots$ until termination:

$$\begin{aligned} w &= A v^{(k-1)} \\ v^{(k)} &= w / \|w\| \\ \lambda^{(k)} &= (v^{(k)})^T A v^{(k)} \quad \text{(Rayleigh Quotient)} \end{aligned}$$

Example 2.2 (Convergence Behavior)**.** Consider diagonal matrices $A = \text{diag}(32, 31, \ldots)$ and $B = \text{diag}(32, 30, \ldots)$. The convergence rate depends on the ratio $|\lambda_2/\lambda_1|$. For A, the ratio is $31/32 \approx 0.97$ (slow). For B, it is $30/32 \approx 0.94$ (faster).

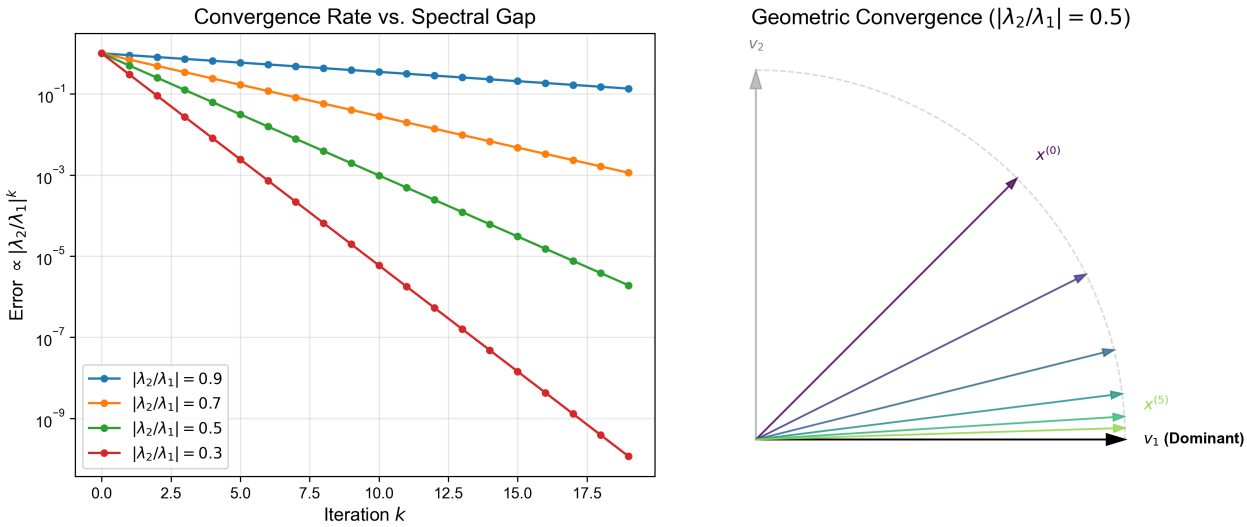

Figure 2: Power Method Convergence. Left: The error decreases linearly with a rate determined by the spectral gap $|\lambda_2/\lambda_1|$. Right: Geometric intuition showing the iteration vector rotating towards the dominant eigenvector v_1.

Observation: The figure above confirms that the error decays linearly on a log scale, with the slope determined by the ratio $|\lambda_2/\lambda_1|$. A smaller ratio (larger gap) yields faster convergence.

2.4 PageRank Revisited: A Detailed Example

Example 2.3 (PageRank on a Toy Network)**.** Let us return to the problem described in the motivation and illustrate how the power method works to yield the PageRank vector. Consider the link structure of the toy network depicted in Figure 3. Here the webpages are nodes in the graph, numbered 1 through 6. Notice that the graph is *directed*.

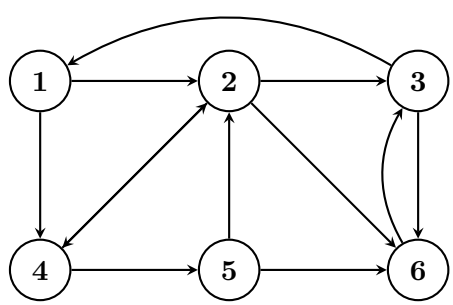

Figure 3: A toy network for the PageRank Example. The connectivity structure determines the importance (rank) of each page.

The task of constructing the link matrix becomes straightforward now. The j-th column represents the outlinks from page j. For instance, the third column of the matrix indicates that node 3 has outlinks to the two nodes numbered 1 and 6. Each of them is thus assigned the value 1/2 in their corresponding location in the column, and the rest of the column entries are set to zero. We obtain:

$$A = \begin{pmatrix} 0 & 0 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 1/3 & 0 & 0 & 0 & 1 \\ 1/2 & 1/3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/2 & 0 & 0 \\ 0 & 1/3 & 1/2 & 0 & 1/2 & 0 \end{pmatrix}$$

In the matrix A, the rows indicate the inlinks. For example, row 2 indicates that the nodes linking to node 2 are numbered 1, 4, and 5. Note that only the columns sum up to 1; we have no control over the sum of elements in any particular row. Such a matrix is called *column stochastic*.

To compute PageRank, we start with the vector $v_0 = (1/6, 1/6, 1/6, 1/6, 1/6, 1/6)^T$ and repeatedly multiply it by A. Eventually, the method converges to:

$$x = \begin{pmatrix} 0.0994 \\ 0.1615 \\ 0.2981 \\ 0.1491 \\ 0.0745 \\ 0.2174 \end{pmatrix}$$

which is the desired PageRank vector. This shows that node 3 is the top-ranked entry.

Potential Difficulties: Can we expect this basic model to always work for a general Internet network? Not without making adjustments. There are two potential difficulties:

1. **Dangling Nodes:** A node with no outlinks (like a PDF file or an image) creates a column of zeros in A. This means the matrix is no longer stochastic, and the "random surfer" gets stuck. The fix is to replace the zero column with a column of $1/n$, implying the surfer jumps to a random page.

2. **Cyclic Paths / Disconnected Components:** A group of nodes might form a trap (a "terminal strong component") where a surfer can enter but never leave. This can prevent convergence to a unique global ranking.

The Solution: Damping Factor To fix these issues, we modify the model. We assume that with probability α (the *damping factor*, typically $\alpha \approx 0.85$), the surfer follows the links, and with probability $1 - \alpha$, they jump to a random page. The modified matrix is:

$$M = \alpha A + (1 - \alpha)\frac{1}{n}\mathbf{1}\mathbf{1}^T$$

This matrix M is strictly positive (all entries > 0), which by the Perron-Frobenius theorem guarantees a unique dominant eigenvector with eigenvalue 1.

2.5 Inverse and Rayleigh Quotient Iterations

The power method converges linearly and its asymptotic error constant is $|\lambda_2/\lambda_1|$. If λ_2 is very close in magnitude to λ_1 (a situation that often occurs in applications), convergence may be extremely slow. Just to illustrate this, if $\lambda_1 = 1$ and $\lambda_2 = 0.99$, then for $k = 100$ we have $(0.99)^{100} \approx 0.36$, which means that after 100 iterations we are not even close to gaining a single decimal digit!

The *inverse iteration* overcomes this difficulty by what is known as a **shift and invert technique**, which gives rise to a substantially faster convergence at the considerable price of having to solve a linear system in each iteration.

2.5.1 Shift and Invert Technique

The idea is as follows. If the eigenvalues of A are λ_j, the eigenvalues of $A - \alpha I$ are $\lambda_j - \alpha$, and the eigenvalues of $B = (A - \alpha I)^{-1}$ are

$$\mu_j = \frac{1}{\lambda_j - \alpha}.$$

Now, the closer α is to λ_1, the more dominant the largest eigenvalue of B is. Indeed, in the limit if $\alpha \to \lambda_1$, then the first eigenvalue of B tends to ∞ while the other eigenvalues tend to finite values. So, suppose we were to apply the power method to $B = (A - \alpha I)^{-1}$ rather than to A, and suppose that λ_2 is the eigenvalue of A that is closest to λ_1. Then such an iteration, though still converging linearly, does so at the improved rate:

$$\left|\frac{\mu_2}{\mu_1}\right| = \left|\frac{1/(\lambda_2 - \alpha)}{1/(\lambda_1 - \alpha)}\right| = \left|\frac{\lambda_1 - \alpha}{\lambda_2 - \alpha}\right|.$$

The closer this number is to zero, the faster the convergence. Indeed, if α is very close to λ_1, convergence is expected to be very fast.

There are two issues to address here:

1. **Choice of parameter α:** We need an α that is easy to compute and sufficiently close to λ_1. For example, $\alpha = \|A\|_1$ or $\alpha = \|A\|_\infty$ can be reasonable choices.

2. **Computational Cost:** In the power method, each iteration involves a matrix-vector product. In inverse iteration, it requires solving a linear system $(A - \alpha I)w = v^{(k-1)}$. This is much more expensive (typically $O(n^3)$ for factorization + $O(n^2)$ per step, vs $O(n^2)$ or $O(nnz)$ for power method). Thus, inverse iteration is most attractive for smaller problems or when fast direct solvers are available.

Definition 2.2 (Algorithm: Inverse Iteration)**.** Input: A, initial guess $v^{(0)}$, shift α. For $k = 1, 2, \ldots$:

$$\begin{aligned} \text{Solve } (A - \alpha I)w &= v^{(k-1)} \\ v^{(k)} &= w/\|w\| \\ \lambda^{(k)} &= (v^{(k)})^T A v^{(k)} \end{aligned}$$

2.5.2 Rayleigh Quotient Iteration

We can improve convergence even further by updating our shift α dynamically. The best estimate for the eigenvalue corresponding to a vector approximation x is given by the Rayleigh quotient $\rho(x) = \frac{x^T A x}{x^T x}$. With this approach, the convergence speed increases as we get closer to the sought eigenvalue; thus the convergence order is better than linear. In fact, in most cases it is *cubic*!

Definition 2.3 (Algorithm: Rayleigh Quotient Iteration)**.** Input: A, initial guess $v^{(0)}$. Set $\lambda^{(0)} = \rho(v^{(0)})$. For $k = 1, 2, \ldots$:

$$\begin{aligned} \text{Solve } (A - \lambda^{(k-1)} I)w &= v^{(k-1)}, \\ v^{(k)} &= w/\|w\|, \\ \lambda^{(k)} &= \rho(v^{(k)}) = \frac{(v^{(k)})^T A v^{(k)}}{(v^{(k)})^T v^{(k)}}. \end{aligned}$$

The SVD can come in handy here. Below we demonstrate use of the SVD in several challenging or otherwise interesting situations.

2.6 Solving almost singular linear systems

Let A be $n \times n$ and real. If $\kappa(A)$ is very large, then solving a linear system $Ax = b$ can be an ill-conditioned problem. Of course, given the SVD of A it is easy to find that $x = V\Sigma^{-1}U^T b$. But this does not make the resulting numerical solution more meaningful. Basically, for an ill-conditioned problem the smallest singular values, although positive, are very small ("almost zero" in some sense). If the problem is too ill-conditioned to be of use, then people often seek to **regularize** it. This means, replace the given problem intelligently by a nearby problem which is better conditioned.

Note: Note the departure made here from everything else done thus far. Instead of seeking numerical algorithms for solving the given problem, we change the problem first and solve a nearby one in a hopefully more meaningful way.

Using SVD this can be done by setting the singular values below a cutoff tolerance to 0, and minimizing the ℓ_2-norm of the solution to the resulting underdetermined problem. We proceed as follows:

1. Starting from n go backward until r is found such that $\frac{\sigma_1}{\sigma_r}$ is tolerable in size. This is the condition number of the problem that we actually solve.
2. Calculate $z = U^T b$; in fact just the first r components of z are needed. In other words, if u_i is the i-th column vector of U, then $z_i = u_i^T b$, $i = 1, \ldots, r$.
3. Calculate $y_i = \sigma_i^{-1} z_i$, $i = 1, 2, \ldots, r$, and set $y_i = 0$, $i = r+1, \ldots, n$.
4. Calculate $x = Vy$. This really involves only the first r columns of V and the first r components of y. In other words, if v_i is the i-th column vector of V, then $x = \sum_{i=1}^{r} y_i v_i$.

Of course the resulting x may not satisfy $Ax = b$ in general. But it's the best one can do under certain circumstances, and it produces a solution x of the smallest norm for a sufficiently well-conditioned approximate problem.

Example 2.4 (Solving a Singular System)**.** Consider

$$A = \begin{pmatrix} 1 & 1 \\ 3 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 6 \end{pmatrix}.$$

Here, A is singular, but b is in its range. So, there are many solutions to the equations $Ax = b$. A straightforward Gaussian elimination, however, results in a division by 0. MATLAB yields the SVD:

$$U \approx \begin{pmatrix} -0.316 & -0.948 \\ -0.948 & 0.316 \end{pmatrix}, \quad V \approx \begin{pmatrix} -0.707 & 0.707 \\ -0.707 & -0.707 \end{pmatrix}, \quad \Sigma \approx \begin{pmatrix} 4.472 & 0 \\ 0 & 4.018 \times 10^{-16} \end{pmatrix}.$$

We decide (wisely) that the smaller singular value is too small, and thus that the effective rank of this matrix is $r = 1$. We then calculate

$$z_1 = -6.324\cdots \to y_1 = -1.414\cdots \to x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Recall that all solutions for this problem have the form $\tilde{x} = (1+\alpha, 1-\alpha)^T$. Since $\|\tilde{x}\|_2^2 = (1+\alpha)^2 + (1-\alpha)^2 = 2(1+\alpha^2)$, we see that our SVD procedure has stably found the solution of minimum ℓ_2-norm for this singular problem.

Let us reemphasize the subtle point that our SVD procedure solves a different problem from the original $Ax = b$, namely, a problem having precisely one solution which generally satisfies the given equations only approximately. The procedure outlined above for solving highly ill-conditioned systems makes use of the Best Lower Rank Approximation Theorem given below. The approximation is known as **truncated SVD**.

Theorem 2.1 (Best Lower Rank Approximation)**.** *The best rank-r approximation A_r of a matrix $A = U\Sigma V^T$, in the sense that $\|A - A_r\|_2 = \sigma_{r+1}$ is at a minimum, is the matrix*

$$A_r = \sum_{i=1}^{r} \sigma_i u_i v_i^T,$$

where u_i and v_i are the i-th column vectors of U and V, respectively.

2.7 Compressing image information

The fact that the best lower rank approximation can be so directly obtained by the SVD makes it possible to devise a *compression scheme*: by storing the first r columns of U and V, as well as the first r singular values, we obtain an approximation of the matrix A using only $r(m + n + 1)$ locations in place of the original mn.

Example 2.5 (Image Compression)**.** Consider the following Python code using `matplotlib` and `numpy`:

```python
import matplotlib.pyplot as plt
import numpy as np

# Load image
img = plt.imread('clown.png')
# Convert to grayscale if necessary...

# Compute SVD
U, S, Vt = np.linalg.svd(img, full_matrices=False)

# Reconstruct with rank r=20
r = 20
img_compressed = U[:, :r] @ np.diag(S[:r]) @ Vt[:r, :]

# Display
plt.imshow(img_compressed, cmap='gray')
```

These instructions load a clown image into an array, find its SVD, and display the image obtained from a rank-20 SVD approximation. **Rank-20 approximation** means we reconstruct the image using only the first 20 singular values and corresponding vectors ($r = 20$ in the summation formula), effectively capturing the most significant features while discarding finer details and noise. The original image is displayed in Figure 4 and the compression result is in Figure 5.

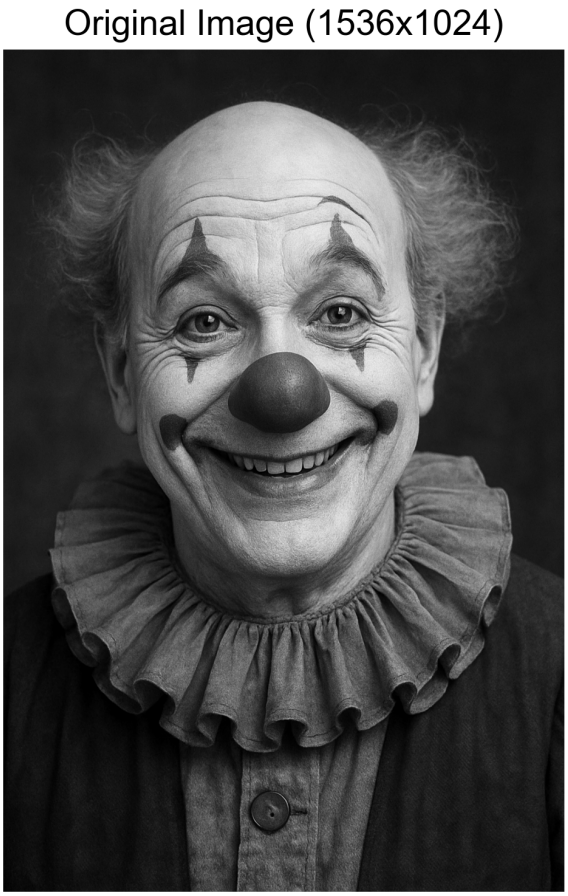

Figure 4: Original image of a clown.

Figure 5: Rank-20 SVD approximation.

The original storage requirements for A are $200 \times 320 = 64,000$, whereas the compressed representation requires $20 \times (200 + 320 + 1) \approx 10,000$ storage locations. This result is certainly better than some random approximation.

2.8 Latent Semantic Analysis (LSA)

In information retrieval, a fundamental challenge is matching a user's query to relevant documents based on meaning rather than just keyword overlap. Simple keyword matching fails when different words describe the same concept (synonymy) or when the same word has multiple meanings (polysemy). Latent Semantic Analysis (LSA) addresses this by uncovering the hidden, or "latent," thematic structure in a collection of text.

The core idea is to construct a *term-document matrix* A, where each row represents a unique word and each column represents a document. The entry A_{ij} corresponds to the frequency (or a weighted frequency) of term i in document j.

Example 2.6 (Toy Data Set for LSA)**.** Consider a simple collection of two documents:

- D1: "Numerical computations are fun."
- D2: "Numerical algorithms and numerical methods are interesting."

After removing common "stop words" (like "are", "and"), our vocabulary is: algorithms, computations, fun, interesting, methods, numerical. The term-document matrix A is:

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 2 \end{pmatrix}.$$

A query like "numerical methods" can be represented as a vector $\mathbf{q} = [0, 0, 0, 0, 1, 1]^T$.

To find relevant documents, we could compute the cosine similarity between the query vector and each document vector (column of A). However, LSA goes a step further. It computes a truncated SVD of A:

$$A_k = U_k \Sigma_k V_k^T$$

By keeping only the largest k singular values, we project the terms and documents into a lower-dimensional "semantic space." In this reduced space, documents that share similar topics will be close to each other, even if they don't share many specific words. This noise reduction technique is powerful for discovering conceptual similarity.

2.9 Rank Deficient Linear Least Squares

In Chapter 6, we solved the least squares problem $\min_{\mathbf{x}} \|\mathbf{b} - A\mathbf{x}\|_2$ assuming A had full column rank. When A is *rank-deficient* (columns are linearly dependent), the solution is not unique. There is an infinite set of vectors $\mathbf{x}$ that minimize the residual.

To resolve this ambiguity, we typically seek the unique solution that has the *minimum Euclidean norm*, $\|\mathbf{x}\|_2$. This is a natural choice as it avoids unnecessary large coefficients in the solution vector.

Using the SVD $A = U\Sigma V^T$, we can explicitly construct this minimum norm solution. If A has rank $r < n$, we define the *pseudo-inverse* $A^\dagger$ as:

$$A^\dagger = V\Sigma^\dagger U^T, \quad \text{where } \Sigma^\dagger_{ii} = \begin{cases} 1/\sigma_i & \text{if } \sigma_i > 0 \\ 0 & \text{if } \sigma_i = 0 \end{cases}.$$

The optimal solution is then $\mathbf{x} = A^\dagger \mathbf{b}$.

Algorithm: SVD Least Squares

1. Compute SVD: $A = U\Sigma V^T$.
2. Determine rank r (count singular values $\sigma_i > \text{tol}$).
3. Compute $\mathbf{z} = U^T\mathbf{b}$.
4. Scale: $y_i = z_i/\sigma_i$ for $i \leq r$, and $y_i = 0$ for $i > r$.
5. Transform: $\mathbf{x} = V\mathbf{y}$.

Example 2.7 (Rank Deficient Least Squares with Python)**.** Let's revisit a least squares problem but introduce rank deficiency by adding a dependent column.

```python
import numpy as np

# Original matrix A (well-conditioned)
A = np.array([[1, 1], [1, 1.000001], [1, 1]])
b = np.array([2, 2, 2])

# Create rank-deficient matrix B by adding a dependent column
# Column 3 is the sum of columns 1 and 2
B = np.column_stack((A, np.sum(A, axis=1)))

# 1. Standard Least Squares (using QR/LAPACK)
# This often warns about rank deficiency or produces large norms
x_qr, residuals, rank, s = np.linalg.lstsq(B, b, rcond=None)
print(f"QR Solution Norm: {np.linalg.norm(x_qr)}")

# 2. SVD-based Minimum Norm Solution (using pinv)
x_svd = np.linalg.pinv(B) @ b
print(f"SVD Solution Norm: {np.linalg.norm(x_svd)}")
```

Listing 1: Solving Rank Deficient LS

The QR-based solver might produce a solution with a large norm due to the lack of uniqueness handling. The SVD-based approach (via `pinv`) guarantees the minimum norm solution, which is often more physically meaningful and stable.

Computational Cost

The SVD is computationally intensive, costing roughly $2mn^2 + 11n^3$ flops. For square matrices, this is significantly more expensive than QR or LU factorization. Therefore, SVD is usually reserved for cases where rank deficiency is expected or when the singular values themselves are needed for analysis.

3 General Methods for Computing Eigenvalues and Singular Values

In Section 8.1, we focused on finding a single eigenpair (usually the dominant one) using the Power Method and its variants. While useful, many applications require finding *all* eigenvalues or singular values of a matrix. In this section, we explore the elegant algorithms designed for this purpose. We focus on methods suitable for dense matrices that fit in memory.

3.1 The Two-Stage Approach

Most modern algorithms for computing eigenvalues or singular values operate in two distinct stages:

1. **Finite Reduction:** The matrix is reduced to a "condensed" form (Hessenberg, Tridiagonal, or Bidiagonal) using a finite sequence of orthogonal similarity transformations. This preserves the eigenvalues (or singular values) while introducing many zeros to simplify subsequent computations.

2. **Iterative Convergence:** An iterative process is applied to the condensed matrix to drive it toward a diagonal or triangular form, from which the values can be easily read.

Why two stages? The iterative part is much cheaper per step when applied to a sparse, condensed matrix ($O(n^2)$ or $O(n)$) compared to a full matrix ($O(n^3)$).

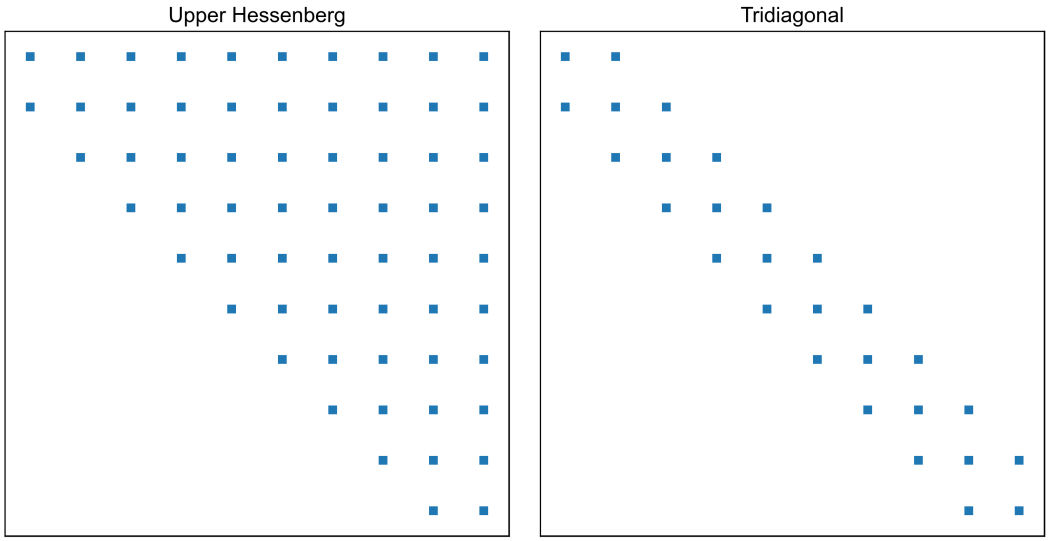

Figure 6: Sparsity patterns after the first stage: Upper Hessenberg form for general non-symmetric matrices (left) and Tridiagonal form for symmetric matrices (right).

3.2 The QR Algorithm for Eigenvalues

The QR algorithm is the workhorse of eigenvalue computations. For a real, square matrix A, the goal is to find the Schur form $T = Q^T AQ$, where T is upper triangular. The diagonal entries of T are the eigenvalues.

Stage 1: Reduction to Hessenberg Form

We cannot jump straight to upper triangular form in a finite number of steps (that would be equivalent to finding roots of a polynomial, which is iterative). However, we *can* reach Upper Hessenberg form (zeros below the first subdiagonal) using Householder reflections.

Example 3.1 (Reduction to Hessenberg Form)**.** Consider a 4×4 matrix. We apply Householder reflectors $P_1, P_2, \ldots$ to zero out columns below the subdiagonal. Crucially, we apply the similarity transformation $P^T AP$. The multiplication on the right (AP) generally destroys zeros created by the multiplication on the left ($P^T A$), *unless* we stop at the subdiagonal.

```python
import numpy as np

def hessenberg_reduction(A):
    """
    Reduces a square matrix A to upper Hessenberg form H = Q^T A Q
    using Householder reflections.
    """
    H = A.copy()
    n = H.shape[0]

    for k in range(n - 2):
        # Vector to annihilate below the subdiagonal
        x = H[k+1:, k]

        # Construct Householder reflector
        e1 = np.zeros_like(x)
        e1[0] = 1.0
```

```
        # Choose sign to avoid cancellation
        alpha = -np.sign(x[0]) * np.linalg.norm(x)
        u = x - alpha * e1
        u = u / np.linalg.norm(u)

        # Apply Q = I - 2uu^T from the left: H = Q^T H
        # (I - 2uu^T) H = H - 2u(u^T H)
        H[k+1:, k:] -= 2.0 * np.outer(u, u.T @ H[k+1:, k:])

        # Apply Q from the right: H = H Q
        # H (I - 2uu^T) = H - 2(H u)u^T
        H[:, k+1:] -= 2.0 * np.outer(H[:, k+1:] @ u, u)

    return H
```

Listing 2: Reduction to Hessenberg Form (Python)

Stage 2: The QR Iteration

Once in Hessenberg form, we apply the QR iteration. The basic idea is surprisingly simple: compute the QR factorization $A_k = Q_k R_k$, then multiply in reverse order $A_{k+1} = R_k Q_k$.

This process creates a sequence of orthogonally similar matrices ($A_{k+1} = Q_k^T A_k Q_k$) that converges to Schur form.

Shifting for Speed: The basic iteration is slow. To accelerate it, we use *shifts*. We subtract a scalar μ_k (an estimate of an eigenvalue) before factoring:

$$\begin{aligned} A_k - \mu_k I &= Q_k R_k \\ A_{k+1} &= R_k Q_k + \mu_k I \end{aligned}$$

A good choice for μ_k is the element in the bottom-right corner, $A_{n,n}$, or the Wilkinson shift (eigenvalue of the bottom-right 2×2 block). This leads to quadratic or even cubic convergence.

```
def qr_algorithm_with_shifts(H, tol=1e-12, max_iter=1000):
    """
    Computes eigenvalues of a Hessenberg matrix H using the QR algorithm
    with explicit shifts.
    """
    A = H.copy()
    n = A.shape[0]
    evals = []

    # Deflation loop: find eigenvalues one by one from the bottom
    for m in range(n, 1, -1):
        iter_count = 0
        while iter_count < max_iter:
            # Check for convergence (subdiagonal element small enough)
            if abs(A[m-1, m-2]) < tol:
                break

            # Simple shift: use the bottom-right element
            shift = A[m-1, m-1]
            I = np.eye(m)

            # QR step with shift
            Q, R = np.linalg.qr(A[:m, :m] - shift * I)
            A[:m, :m] = R @ Q + shift * I

            iter_count += 1

```

```
        # Store found eigenvalue and deflate
        evals.append(A[m-1, m-1])
        # (In a real implementation, we would decouple the matrix here)

    evals.append(A[0, 0]) # Last remaining eigenvalue
    return np.array(evals)
```

Listing 3: QR Iteration with Shifts (Python)

Example 3.2 (QR Convergence)**.** Applying this algorithm to our previous 4×4 example, the subdiagonal entries rapidly converge to zero, revealing the eigenvalues on the diagonal. The use of shifts transforms a slow linear convergence into a fast quadratic one.

Example 3.3 (Differential Equation Eigenvalues)**.** Consider the boundary value problem $u'' - u' = \lambda u$ on $[0, L]$. Discretizing this gives a non-symmetric tridiagonal matrix. For small L (e.g., $L = 10$), our simple QR code works well, finding real eigenvalues that match the analytical solution. However, for large L (e.g., $L = 80$), the problem becomes highly non-normal. The eigenvalues are extremely sensitive to perturbations. A naive QR algorithm might fail or produce complex eigenvalues due to numerical instability, even if the true spectrum is real. This highlights the need for robust, industrial-strength solvers (like LAPACK's `dgeev`) that use implicit shifts and handle deflation carefully.

3.3 Computing the SVD

Computing the Singular Value Decomposition ($A = U\Sigma V^T$) follows a similar two-stage philosophy but requires different transformations on the left and right ($U^T AV$).

1. **Bidiagonal Reduction:** We use Householder reflectors to zero out entries below the diagonal (using U) and entries to the right of the superdiagonal (using V). This yields a bidiagonal matrix B.

2. **Diagonalization:** We apply an iterative variant of the QR algorithm (often the Golub-Kahan-Reinsch algorithm) to B to reduce the superdiagonal elements to zero. The diagonal entries converge to the singular values.

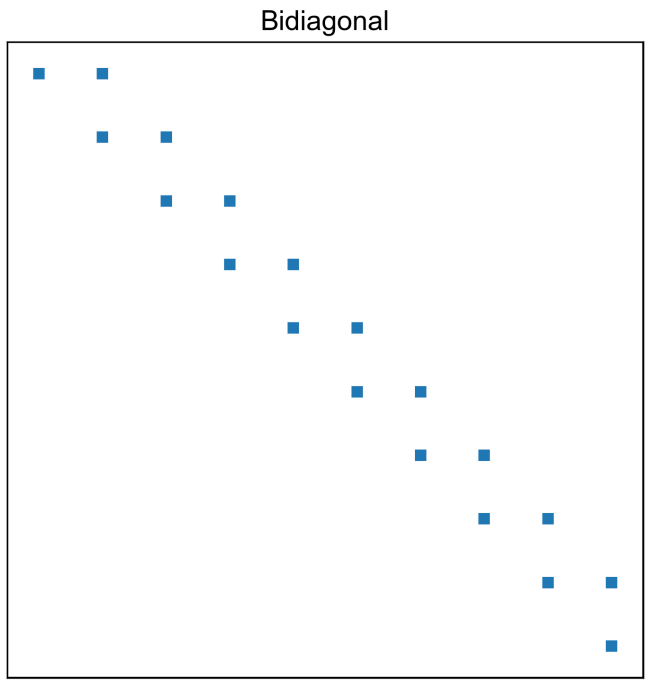

Figure 7: Sparsity pattern after the first stage of SVD computation: A bidiagonal matrix (left). Implicitly, this corresponds to a tridiagonal matrix for $A^T A$ (right).

This approach is numerically stable and ensures that the computed singular values are always non-negative.

Practice Problems

1. Prove that the power method converges linearly with rate $|\lambda_2/\lambda_1|$ when $|\lambda_1| > |\lambda_2|$.
2. Show that the Rayleigh quotient $\rho(x) = \frac{x^T A x}{x^T x}$ minimizes $\|Ax - \lambda x\|_2$ over all scalars λ.
3. **Computational Exercise: Image Compression and SVD Analysis.**
   Load a high-resolution grayscale image (e.g., `clown.png` or a photo of your choice) and convert it to a matrix A.
   (a) **SVD Computation:** Compute the full Singular Value Decomposition of A. Plot the singular values on a semi-log scale. What does the decay rate tell you about the potential for compression?
   (b) **Compression Levels:** Reconstruct the image using the top k singular values for a range of k (e.g., $k \in \{5, 20, 50, 100\}$). Display the reconstructed images side-by-side with the original.
   (c) **Error Analysis:** For each k, compute the relative reconstruction error in the Frobenius norm, $E_k = \frac{\|A - A_k\|_F}{\|A\|_F}$. Plot E_k against k. Verify theoretically that this matches the ratio of the square root of the sum of squared tail singular values.
   (d) **Compression Ratio:** Calculate the theoretical storage savings for each k. At what value of k does the storage cost of the compressed form (U_k, Σ_k, V_k) exceed the storage of the original matrix?
   (e) **Performance Analysis:** Measure the wall-clock time required to compute the SVD for images of increasing resolution (resize the original image to $N \times N$ for $N = 100, 200, \ldots, 2000$). Plot the time vs. N on a log-log scale to estimate the algorithmic complexity (slope of the line).

References

- Ascher, U. M., & Greif, C. (2011). *A First Course in Numerical Methods*. SIAM.
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.